



به نام خدا

مصطفی چرکزی

آزمایش دوم پردازش سیگنال های دیجیتال

آزمایش ۲: تحلیل سیگنال‌های زمان گسسته در حوزه زمان

در این آزمایش، بررسی خواص سیستم‌ها از لحاظ خطی بودن، پایداری، تغییرناپذیری با زمان، علی بودن و ... انجام خواهد شد.

در این قسمت با دستورات `impz`, `filter` و `conv` آشنا می‌شویم.

دستور `impz`، پاسخ ضربه را محاسبه می‌کند: ورودی یک ضربه است.

دستور `conv`، کانولوشن دو سیگنال را محاسبه می‌کند: همچنین دستور `deconv` عکس کانولوشن را بر می‌گرداند (سیگنال اصلی).

$$H(z) = \frac{b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3} + \dots}{1 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3} + \dots}$$

$$H(z) = \frac{2z^{-1} + 3z^{-2} + 5}{1 + 4z^{-1} - 6z^{-2} + z^{-3}} \Rightarrow \text{filter}([5, 2, 3], [1, 4, -6, 1], x)$$

بخش ۱: بررسی فیلتر میانگین یا هموار ساز

فعالیت ۱: تابع فیلتر میانگین در زیر نشان داده شده است.

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k]$$

ابتدا یک سیگنال سینوسی گسسته که یک سیگنال سینوسی با فرکانس بالا بعنوان نویز روی آن سوار شده است، را ایجاد کرده، سپس این سیگنال را بعنوان ورودی به سیستم فیلتر میانگین با رابطه فوق اعمال کرده و ورودی و خروجی را به ازای طول پنجره فیلتر (M) مختلف ترسیم کنید.

```
N = 100;
n = 0:N-1;
f_signal = 0.02;
f_noise = 0.3;

x = sin(2*pi*f_signal*n) + 0.3*sin(2*pi*f_noise*n);

M_values = [3, 5, 10];

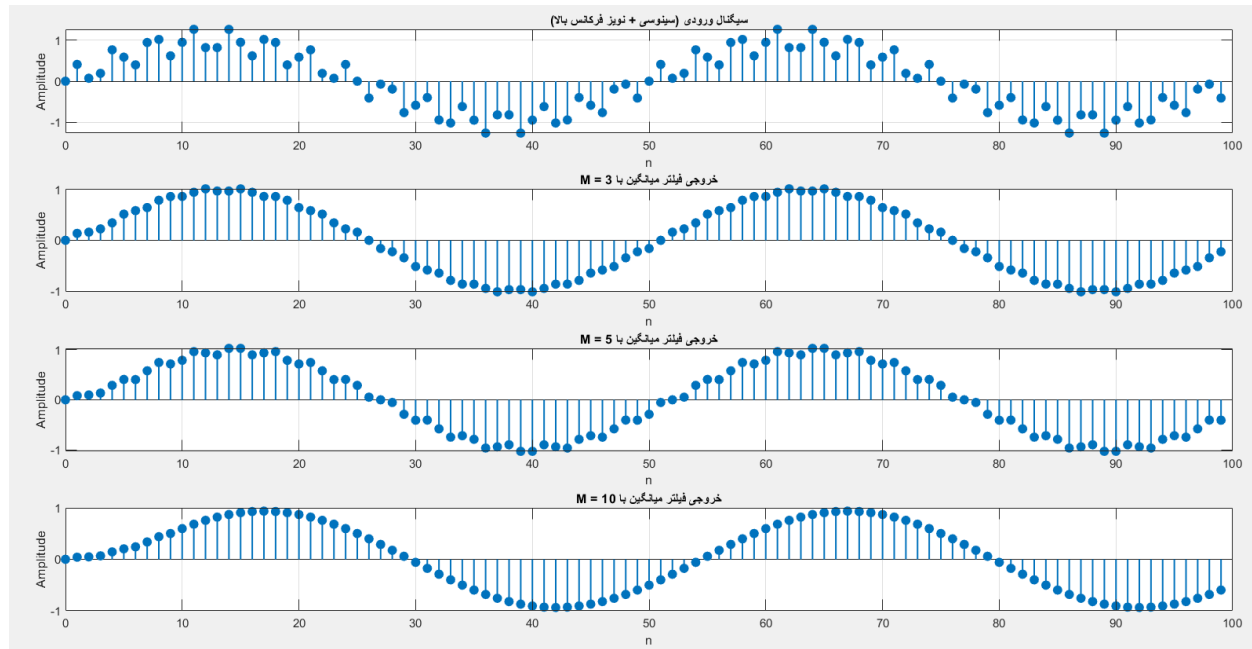
figure;
subplot(length(M_values)+1, 1, 1);
stem(n, x, 'filled', 'LineWidth', 1.2);
title('سیگنال ورودی (سینوسی + نویز فرکانس بالا)');
xlabel('n'); ylabel('Amplitude');
grid on;

for idx = 1:length(M_values)
    M = M_values(idx);
    b = ones(1, M) / M;
    a = 1;
    y = filter(b, a, x);
```

```

subplot(length(M_values)+1, 1, idx+1);
stem(n, y, 'filled', 'LineWidth', 1.2);
title(['خروجی فیلتر میانگین با ' , num2str(M)]);
xlabel('n'); ylabel('Amplitude');
grid on;
end

```



فیلتر میانگین یک فیلتر پایین‌گذر است که نویز فرکانس بالا را حذف می‌کند.

هرچه M بزرگتر باشد، نویز بیشتری حذف می‌شود اما سیگنال هموارتر و لبه‌ها از بین می‌روند.

با M خیلی بزرگ، سیگنال اصلی نیز تضعیف می‌شود.

فعالیت ۲: فعالیت ۱ را برای $M = 2$ و فیلتر زیر تغییر دهید.

$$y[n] = 0.5(x[n] - x[n-1])$$

چه تاثیری بر سیگنال ورودی خواهد داشت؟

```

N = 100;
n = 0:N-1;
f_signal = 0.02;
f_noise = 0.3;

x = sin(2*pi*f_signal*n) + 0.3*sin(2*pi*f_noise*n);

% y[n] = 0.5*(x[n] - x[n-1])
b_diff = [0.5, -0.5];
a_diff = 1;
y_diff = filter(b_diff, a_diff, x);

```

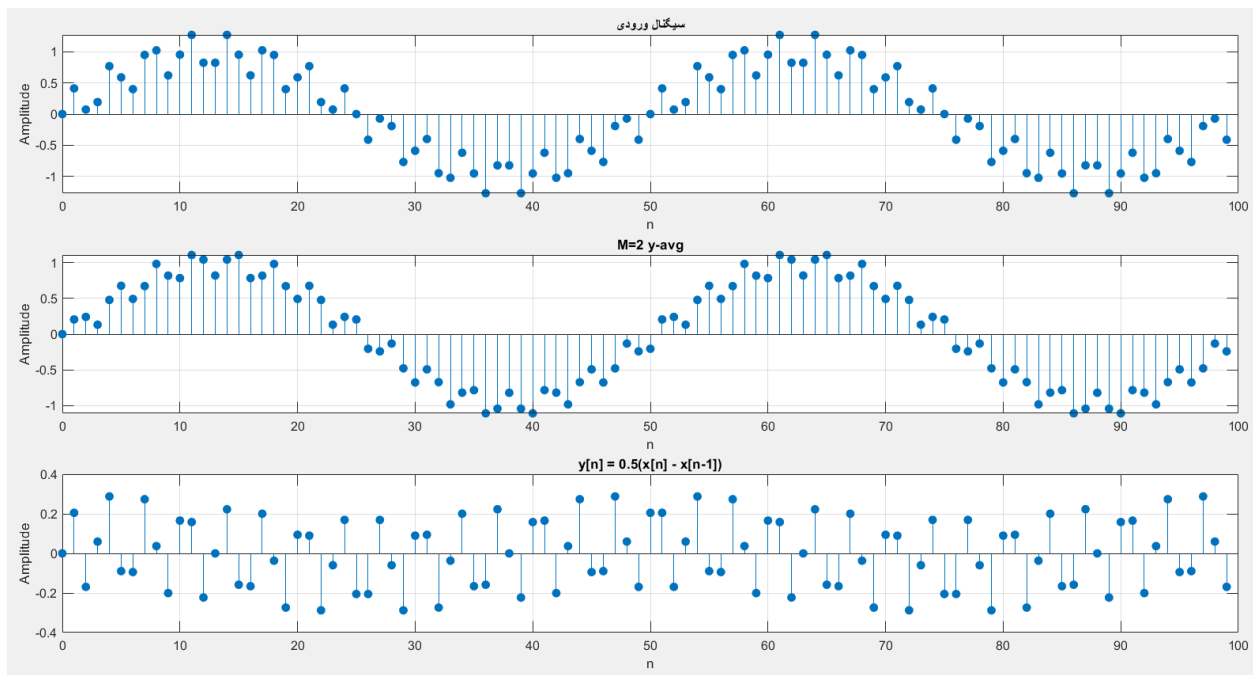
```

figure;
subplot(3,1,1);
stem(n, x, 'filled');
title('سیگنال ورودی');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(3,1,2);
b_avg = [0.5, 0.5];
y_avg = filter(b_avg, 1, x);
stem(n, y_avg, 'filled');
title('M=2 y-avg');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(3,1,3);
stem(n, y_diff, 'filled');
title('y[n] = 0.5(x[n] - x[n-1])');
xlabel('n'); ylabel('Amplitude');
grid on;

```



فیلتر $y[n] = 0.5(x[n] - x[n-1])$ یک فیلتر بالاگذر است. این فیلتر تغییرات سریع را تقویت میکند و تغییرات کند را تضعیف میکند و همچنین فیلتر میانگین یک فیلتر پایین گذر است که فرکانس های بالا را تضعیف میکند.

سیگنال ورودی را با یک سیگنال سینوسی که فرکانس افزایش و متغیر دارد جایگزین کنید: طول سیگنال 0-100 و فرکانس 0.5-0، سپس قسمت فعالیت ۱ و فعالیت ۲ را دوباره با این ورودی جدید اجرا کنید.

```

N = 100;

n = 0:N-1;

a = 0.0025;
b = 0;
arg = 2*pi * (a * n.^2 + b * n);
x_chirp = cos(arg);

```

```

M = 5;
b_avg = ones(1, M)/M;
y_avg_chirp = filter(b_avg, 1, x_chirp);

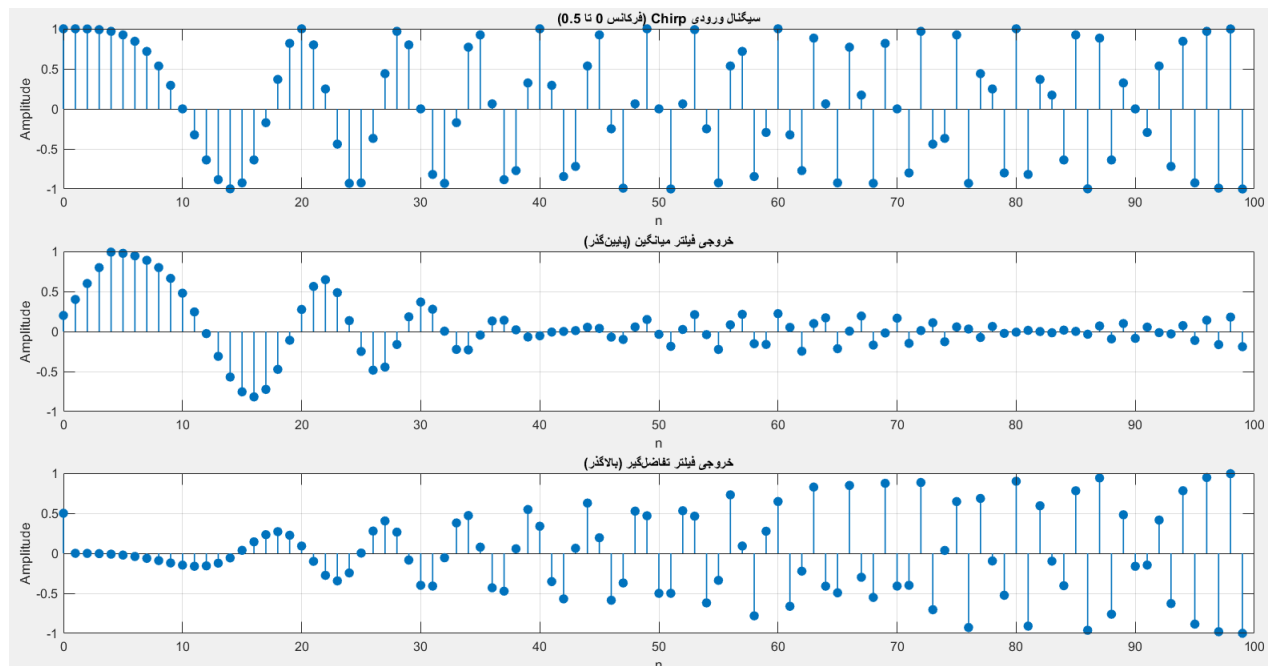
b_diff = [0.5, -0.5];
y_diff_chirp = filter(b_diff, 1, x_chirp);

figure;
subplot(3,1,1);
stem(n, x_chirp, 'filled', 'LineWidth', 1);
title('سیگنال ورودی (فرکانس 0 تا 0.5)');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(3,1,2);
stem(n, y_avg_chirp, 'filled', 'LineWidth', 1);
title('خروجی فیلتر میانگین (پایین‌گذر)');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(3,1,3);
stem(n, y_diff_chirp, 'filled', 'LineWidth', 1);
title('خروجی فیلتر تفاضل‌گیر (بالاگذر)');
xlabel('n'); ylabel('Amplitude');
grid on;

```



بخش ۲: بررسی یک سیستم غیرخطی ساده

فعالیت ۳: سیگنال غیرخطی زیر را در نظر بگیرید:

با استفاده از اعمال ورودی‌های سینوسی با فرکانس‌های مختلف، خروجی را مشاهده نمایید.

سیگنال خروجی چگونه با تغییر فرکانس تغییر میکند؟ اط لحاظ تئوری نیز بررسی کنید.

$$y[n] = x[n]^2 - x[n-1]x[n+1]$$

یک مقدار DC به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

یک سینوسی به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

یک سینوسی به همراه یک مقدار DC به سیستم فوق اعمال کرده و خروجی را بررسی کنید.

نتیجه سه بخش فوق را در جهت بررسی خطی یا غیر خطی بودن سیستم بررسی کنید. (آیا جمع پاسخ دو حالت اول و دوم با حاصل قسمت سوم یکی است؟)

```
N = 100;  
n = 0:N-1;  
  
x_dc = 2 * ones(1, N);  
y_dc = zeros(1, N);  
for i = 2:N-1  
    y_dc(i) = x_dc(i)^2 - x_dc(i-1)*x_dc(i+1);  
end  
  
figure;  
subplot(3,2,1);  
stem(n, x_dc, 'filled');  
title('DC ورودی');  
grid on;  
subplot(3,2,2);  
stem(n, y_dc, 'filled');  
title('DC خروجی برای ورودی');  
grid on;  
  
f = 0.05;  
x_sin = cos(2*pi*f*n);  
y_sin = zeros(1, N);  
for i = 2:N-1  
    y_sin(i) = x_sin(i)^2 - x_sin(i-1)*x_sin(i+1);  
end  
  
subplot(3,2,3);  
stem(n, x_sin, 'filled');  
title('ورودی سینوسی');  
grid on;  
subplot(3,2,4);  
stem(n, y_sin, 'filled');  
title('خروجی برای ورودی سینوسی');  
grid on;  
  
x_comb = x_dc + x_sin;  
y_comb = zeros(1, N);
```

```

for i = 2:N-1
    y_comb(i) = x_comb(i)^2 - x_comb(i-1)*x_comb(i+1);
end

subplot(3,2,5);
stem(n, x_comb, 'filled');
title('ورودی ترکیبی (DC + سینوسی)');
grid on;
subplot(3,2,6);
stem(n, y_comb, 'filled');
title('خروجی برای ورودی ترکیبی');
grid on;

% y_comb = y_dc + y_sin ?
y_sum = y_dc + y_sin;
difference = y_comb - y_sum;

figure;
stem(n, difference, 'filled');
title('y_{DC} + y_{sin}');
xlabel('n'); ylabel('Difference');
grid on;

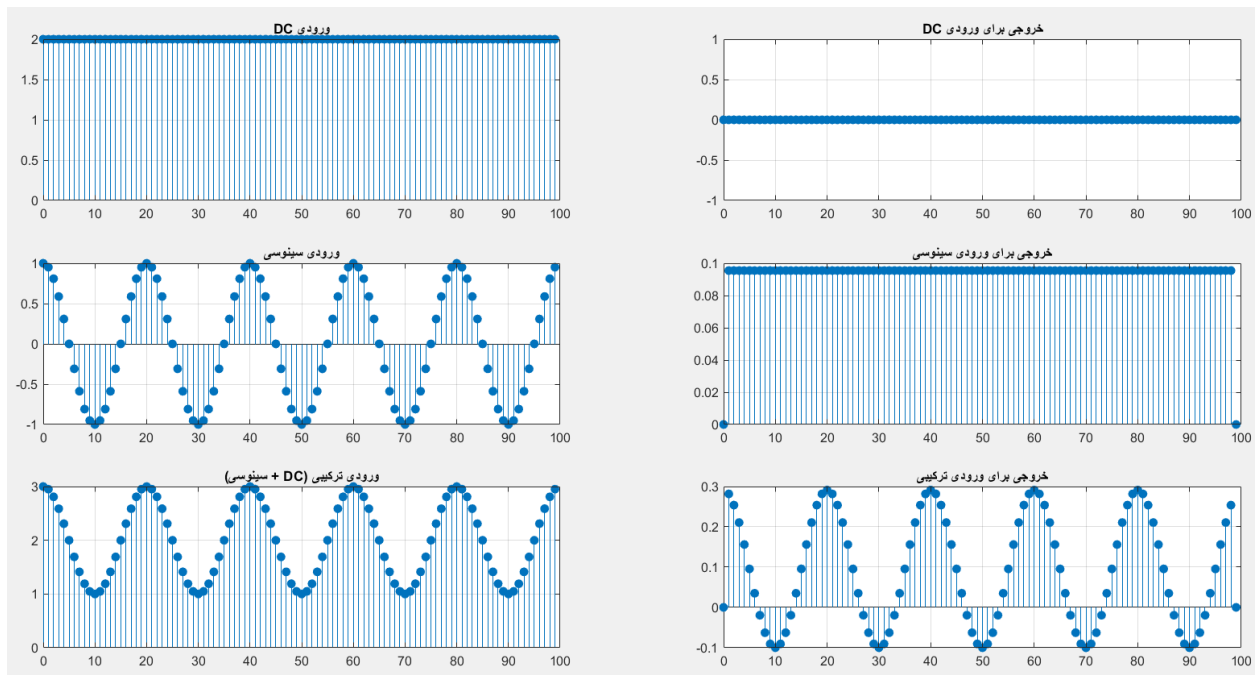
disp(['حداکثر تفاوت: ', num2str(max(abs(difference)))]);

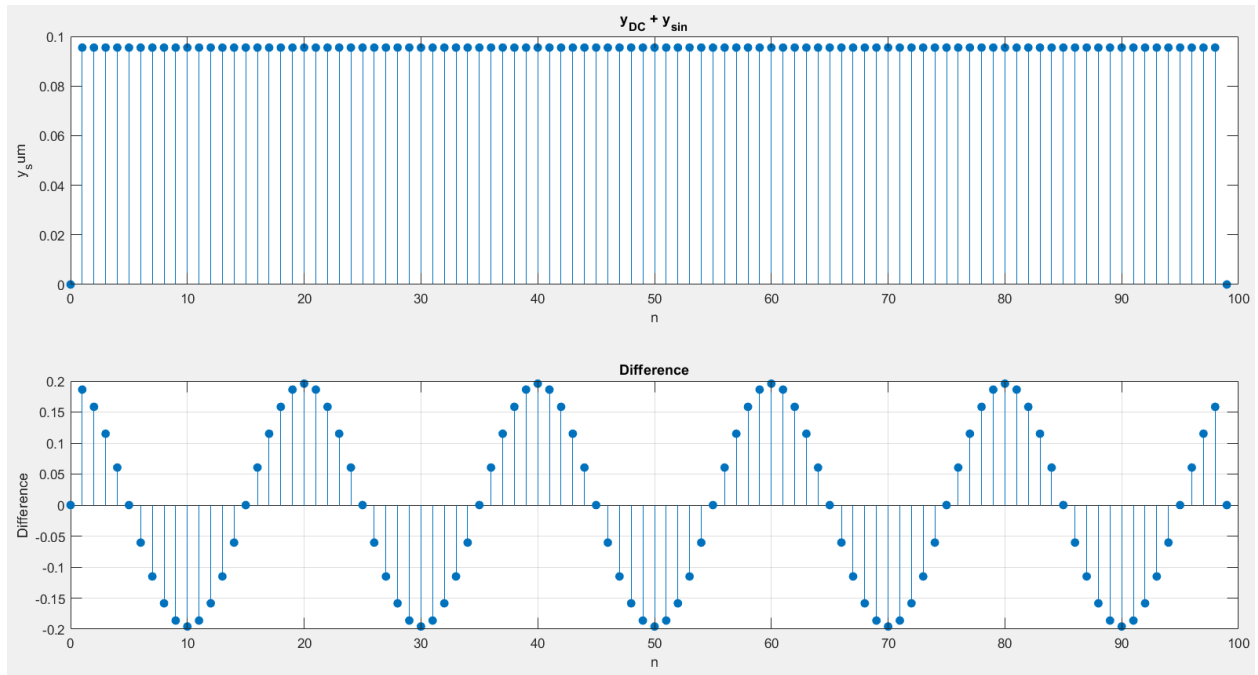
```

در ورودی DC: خروجی صفر میشود

در ورودی سینوسی: خروجی شامل هارمونیک های فرکانس دوبرابر است

در ترکیب سینوسی و DC خروجی ترکیب برابر با مجموع حالت های ۱ و ۲ نیست (یعنی سیستم غیر خطی است)





همانطور که در نمودار بالا مشخص است مجموع خروجی حالت اول و دوم برابر با حالت سوم نیست.

فعالیت ۴: سیستم زیر را از لحاظ خطی بودن بررسی کنید. (اعمال ترکیب خطی از دو ورودی به سیستم و مقایسه با ترکیب خطی خروجی‌ها در صورت اعمال ورودی‌ها بطور مستقل و مقایسه نتیجه)

$$y[n] - 0.4y[n-1] - 0.75y[n-2] = 2.24x[n] + 2.49x[n-1] + 2.24x[n-2]$$

```

N = 100;
n = 0:N-1;

b = [2.24, 2.49, 2.24];
a = [1, -0.4, -0.75];

x1 = cos(2*pi*0.05*n);
x2 = cos(2*pi*0.15*n);

y1 = filter(b, a, x1);
y2 = filter(b, a, x2);

alpha = 2;
beta = 3;
x_combined = alpha*x1 + beta*x2;
y_combined = filter(b, a, x_combined);

y_sum = alpha*y1 + beta*y2;

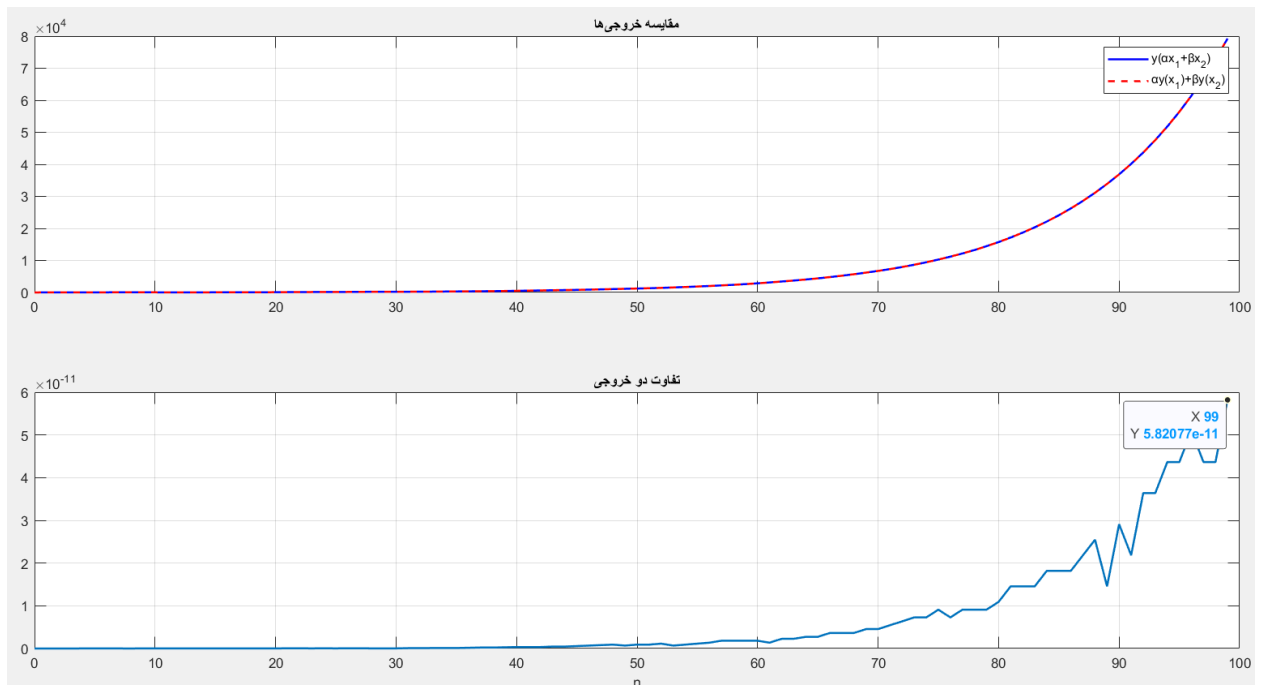
difference = y_combined - y_sum;

figure;
subplot(2,1,1);
plot(n, y_combined, 'b', n, y_sum, 'r--', 'LineWidth', 1.5);
legend('y(αx1+βx2)', 'αy(x1)+βy(x2)');
title('مقایسه خروجی‌ها');
grid on;

```



```
subplot(2,1,2);
plot(n, difference, 'LineWidth', 1.5);
title('تفاوت دو خروجی');
xlabel('n');
grid on;
```



تفاوت دو سیگنال همانطور که مشخص است خیلی کم (نزدیک به صفر = $6e-11$) است بنابراین سیستم خطی است.

فعالیت ۵: سیستم زیر را نیز از لحاظ خطی بودن بررسی کنید.

$$y[n] = x[n] \times x[n-1]$$

```
N = 50;
n = 0:N-1;

x1 = ones(1, N);
x2 = cos(2*pi*0.1*n);
alpha = 1;
beta = 1;

y1 = zeros(1, N);
y2 = zeros(1, N);
for i = 2:N
    y1(i) = x1(i) * x1(i-1);
    y2(i) = x2(i) * x2(i-1);
end

x_comb = alpha*x1 + beta*x2;
y_comb = zeros(1, N);
for i = 2:N
    y_comb(i) = x_comb(i) * x_comb(i-1);
end
```

```

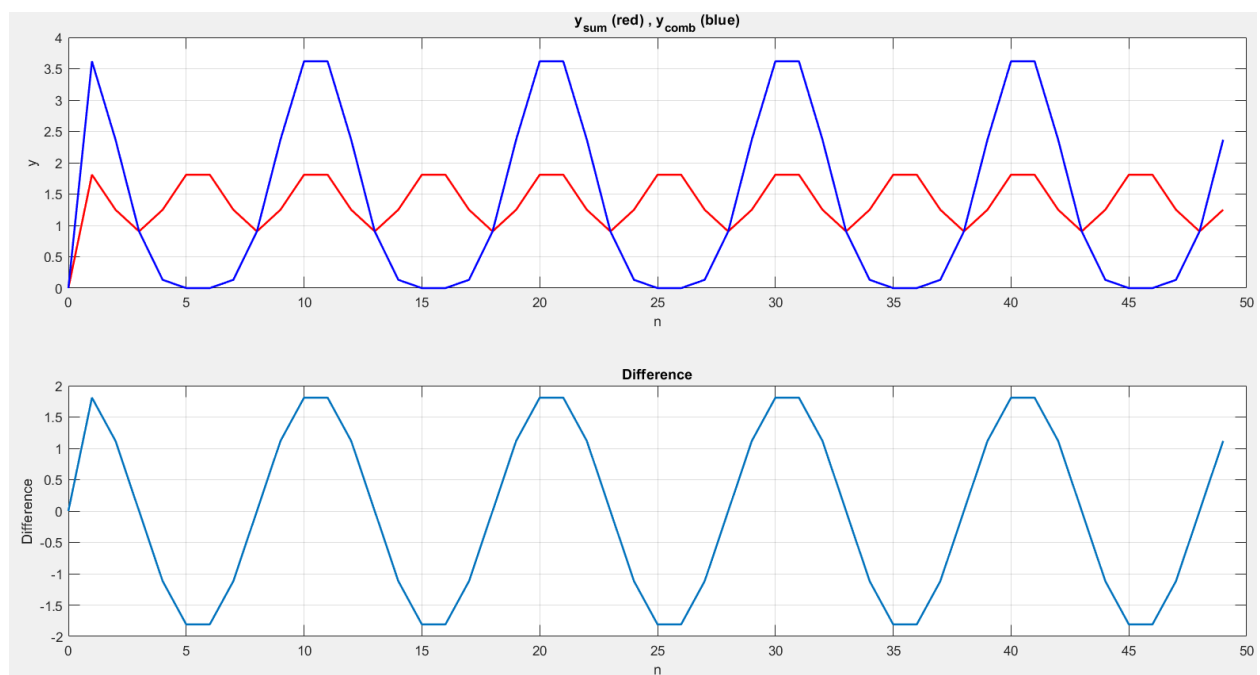
y_sum = alpha*y1 + beta*y2;

% مقایسه
difference = y_comb - y_sum;

figure;
subplot(2,1,1);
plot(n, y_sum, 'r', 'LineWidth', 1.5);
title('y_{sum} (red) , y_{comb} (blue)');
xlabel('n'); ylabel('y');
hold on;
plot(n, y_comb, 'b', 'LineWidth', 1.5);
xlabel('n'); ylabel('y');

grid on;
subplot(2,1,2);
plot(n, difference, 'LineWidth', 1.5);
title('Difference');
xlabel('n'); ylabel('Difference');
grid on;

```



همانطور که مشخص است سیستم غیر خطی است.

فعالیت ۶: در دو فعالیت قبل، در حالت شرایط اولیه غیر صفر نیز نتیجه را بررسی کنید. چه نتیجه‌ای میگیرید؟

```

N = 100;
n = 0:N-1;

b = [2.24, 2.49, 2.24];
a = [1, -0.4, -0.75];

x1 = cos(2*pi*0.05*n);
x2 = cos(2*pi*0.15*n);

```

```

Zi = randn(2, 1);

y1_ic = filter(b, a, x1, Zi);
y2_ic = filter(b, a, x2, Zi);

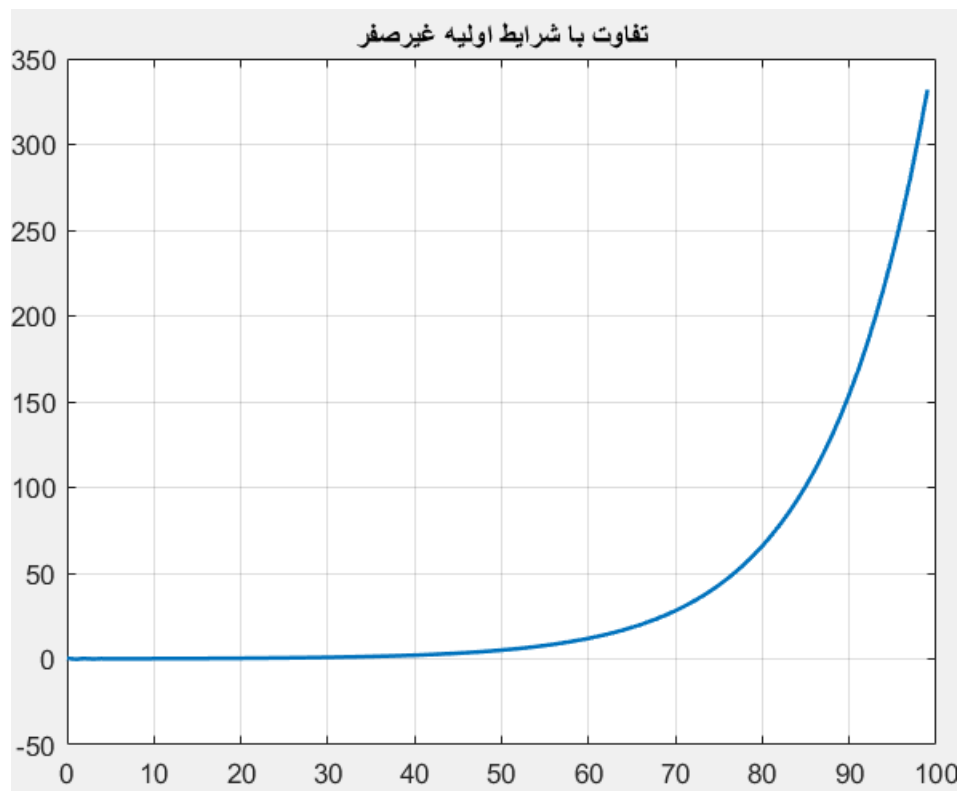
x_comb = x1 + x2;
y_comb_ic = filter(b, a, x_comb, Zi);

y_sum_ic = y1_ic + y2_ic;

difference = y_comb_ic - y_sum_ic;

figure;
plot(n, difference, 'LineWidth', 1.5);
title('تفاوت با شرایط اولیه غیر صفر');
grid on;

```



با شرایط اولیه غیر صفر، تفاوت ایجاد می‌شود. سیستم فقط با شرایط اولیه صفر خطی است. شرایط اولیه غیر صفر خاصیت خطی بودن را نقض می‌کند (پاسخ ورودی صفر + پاسخ حالت صفر).

بخش ۳: بررسی خاصیت تغییرپذیری یا تغییرناپذیری با زمان

فعالیت ۷: سیستم مطرح شده زیر را از لحاظ TI بودن بررسی کنید.

$$y[n] - 0.4y[n-1] - 0.75y[n-2] = 2.24x[n] + 2.49x[n-1] + 2.24x[n-2]$$

برای این کار باید ورودی‌های $x[n]$ و $x[n-D]$ را به سیستم اعمال کرده و خروجی‌های متناظر $y_1[n]$ و $y_2[n]$ را بدست آورده و سپس بررسی کنید که آیا $y_2[n+D]$ با $y_1[n]$ برابر است یا خیر.

این کار را برای مقادیر مختلف از D تکرار کنید و نتیجه را نمایش دهید.

```
N = 100;
n = 0:N-1;

b = [2.24, 2.49, 2.24];
a = [1, -0.4, -0.75];

x = cos(2*pi*0.05*n);

D = 10;
x_delayed = [zeros(1, D), x(1:end-D)];

y_original = filter(b, a, x);
y_delayed_input = filter(b, a, x_delayed);

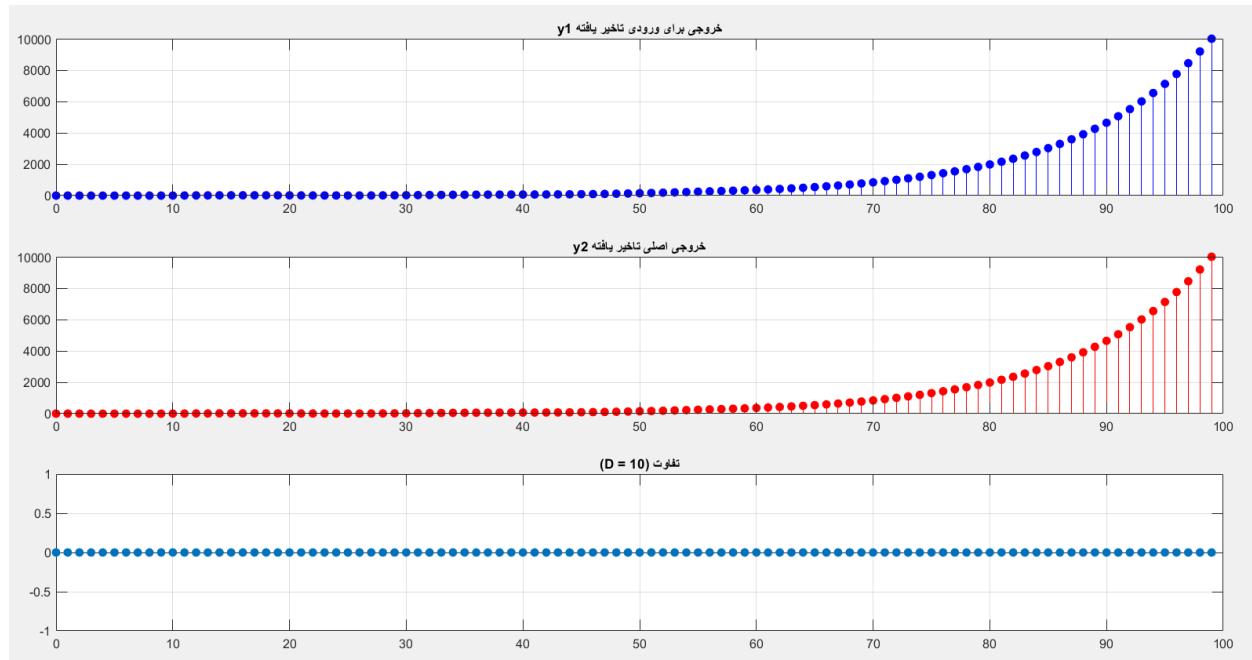
y_original_delayed = [zeros(1, D), y_original(1:end-D)];

difference = y_delayed_input - y_original_delayed;

figure;
subplot(3,1,1);
stem(n, y_delayed_input, 'b', 'filled');
title('y1 خروجی برای ورودی تاخیر یافته');
grid on;

subplot(3,1,2);
stem(n, y_original_delayed, 'r', 'filled');
title('y2 خروجی اصلی تاخیر یافته');
grid on;

subplot(3,1,3);
stem(n, difference, 'filled');
title(['D = ', num2str(D), ' تفاوت']);
grid on;
```



تفاوت ناچیز است $\approx$ صفر بنابراین سیستم تغییرناپذیر با زمان (TI) است.

فعالیت ۸: فعالیت قبل را برای حالت، شرایط اولیه غیر صفر نیز انجام دهید. آیا در اینصورت سیستم تغییرناپذیر با زمان است؟

```
N = 100;
n = 0:N-1;
D = 10;

b = [2.24, 2.49, 2.24];
a = [1, -0.4, -0.75];

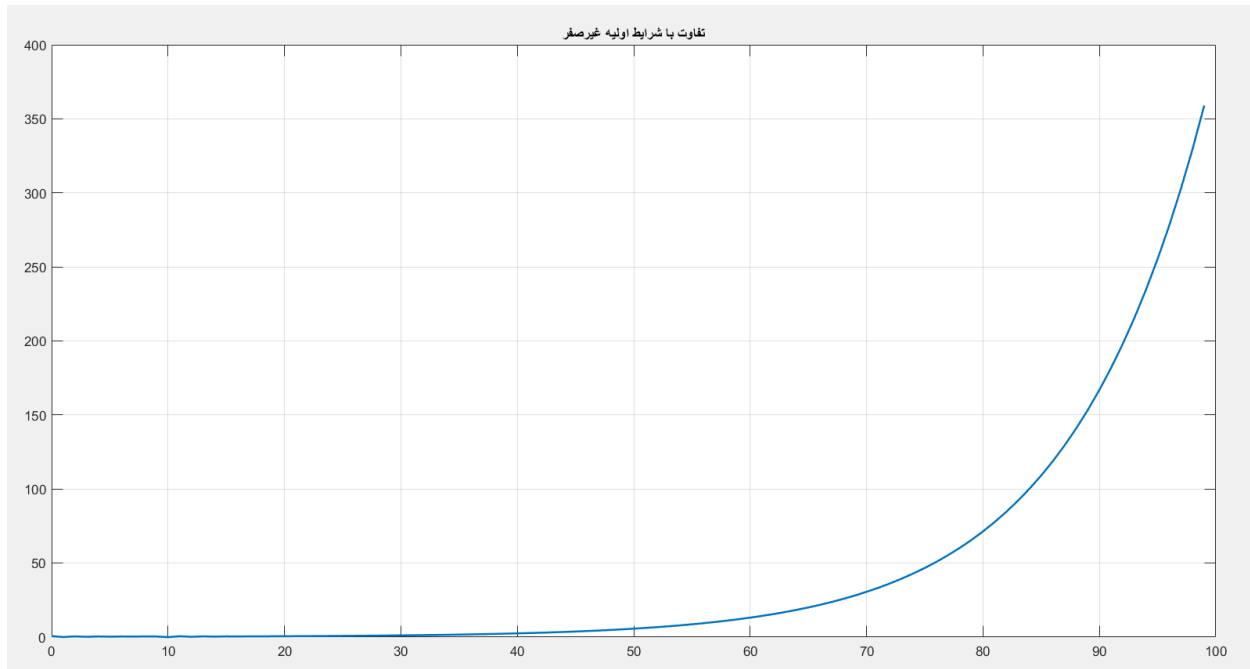
x = cos(2*pi*0.05*n);
x_delayed = [zeros(1, D), x(1:end-D)];

Zi = [0.5; -0.3]; % شرایط اولیه غیر صفر

y_original_ic = filter(b, a, x, Zi);
y_delayed_input_ic = filter(b, a, x_delayed, Zi);
y_original_delayed_ic = [zeros(1, D), y_original_ic(1:end-D)];

difference_ic = y_delayed_input_ic - y_original_delayed_ic;

figure;
plot(n, difference_ic, 'LineWidth', 1.5);
title('تفاوت با شرایط اولیه غیر صفر');
grid on;
```



با شرایط اولیه غیرصفر، تفاوت ایجاد می‌شود که یعنی تغییرناپذیری با زمان فقط برای شرایط اولیه صفر برقرار است.

فعالیت ۹: سیستم قبل را با سیستم زیر جایگزین کرده و خاصیت تغییرناپذیری با زمان را بررسی کنید.

$$y[n] = nx[n] + x[n - 1]$$

```
N = 50;
n = 0:N-1;
D = 10;

x = cos(2*pi*0.1*n);
x_delayed = [zeros(1, D), x(1:end-D)];

% خروجی برای ورودی اصلی
y_original = zeros(1, N);
for i = 2:N
    y_original(i) = n(i)*x(i) + x(i-1);
end

% خروجی برای ورودی تاخیر یافته
y_delayed = zeros(1, N);
for i = 2:N
    y_delayed(i) = n(i)*x_delayed(i) + x_delayed(i-1);
end

% تاخیر خروجی اصلی
y_original_shifted = [zeros(1, D), y_original(1:end-D)];

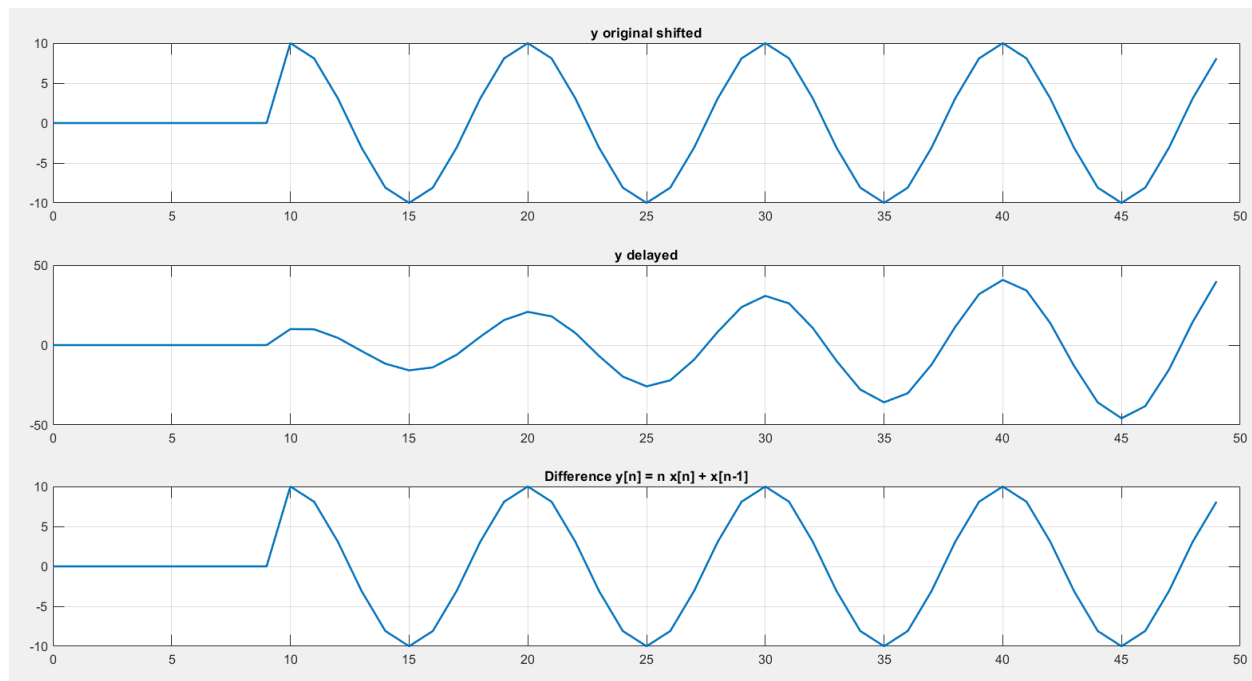
difference = y_delayed - y_original_shifted;

figure;
subplot(3,1,1);
plot(n, difference, 'LineWidth', 1.5);
title('y original shifted');
```

```

grid on;
subplot(3,1,2);
plot(n, y_delayed, 'LineWidth', 1.5);
title('y delayed');
grid on;
subplot(3,1,3);
plot(n, difference, 'LineWidth', 1.5);
title('Difference y[n] = n x[n] + x[n-1]');
grid on;

```



تفاوت زیاد است که یعنی این سیستم تغییرپذیر با زمان (Time-Varying) است.
 علت: به دلیل وجود ضربدر n (زمان) در ورودی x (خروجی به زمان بستگی دارد)

بخش ۴: محاسبه پاسخ ضربه

فعالیت ۱۰: پاسخ ضربه سیستم زیر را با استفاده از دستور `impz(num,den,N)` محاسبه کرده و ۵۰ سمپل اولیه-ی آن را نمایش دهید.

$$y[n] + 0.71y[n-1] - 0.46y[n-2] - 0.62y[n-3] = 0.9x[n] - 0.45x[n-1] + 0.35x[n-2] + 0.2x[n-3]$$

سپس بجای استفاده از دستور `impz` از دستور `filter` پاسخ ضربه‌ی سیستم را محاسبه کنید و ۴۰ سمپل اول را نمایش دهید. شکل خروجی را با قسمت قبل مقایسه کنید.

% ضرایب سیستم

```
b = [0.9, -0.45, 0.35, 0.2];
```

```
a = [1, 0.71, -0.46, -0.62];
```

```

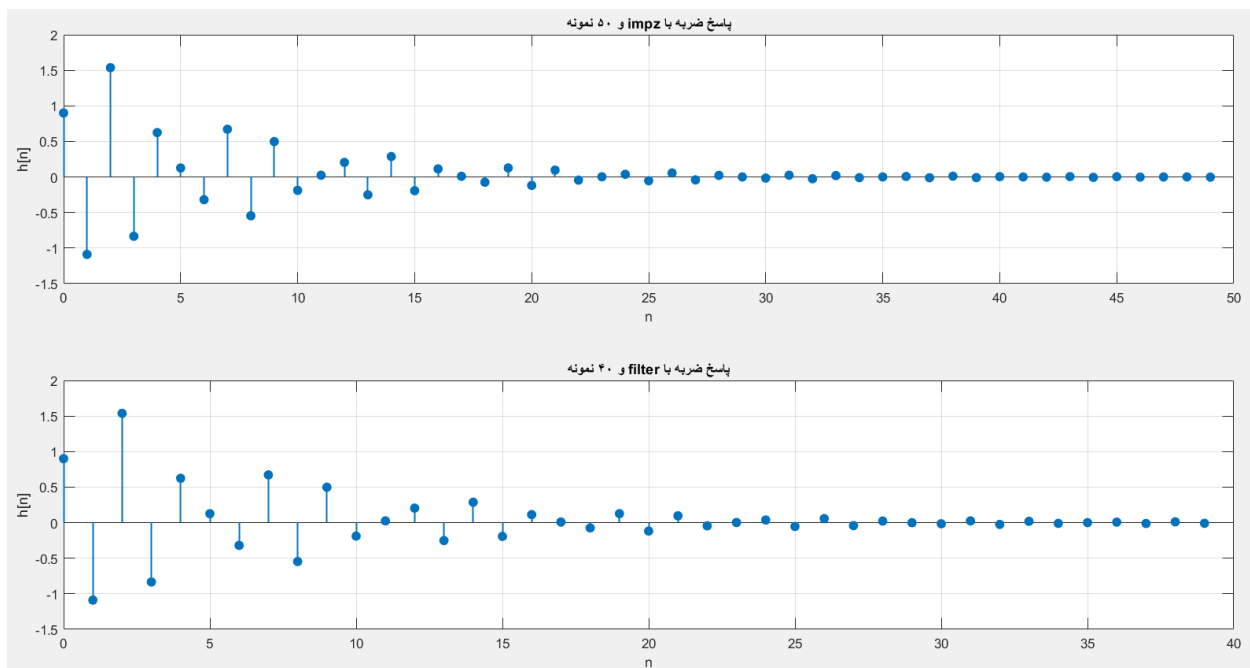
% impz
N_imp = 50;
[h_imp, t_imp] = impz(b, a, N_imp);

% filter
N_filt = 40;
delta = [1, zeros(1, N_filt-1)]; % ورودی ضربه
h_filt = filter(b, a, delta);

% رسم
figure;
subplot(2,1,1);
stem(0:N_imp-1, h_imp, 'filled', 'LineWidth', 1.2);
title('۵۰ نمونه impz پاسخ ضربه با');
xlabel('n'); ylabel('h[n]');
grid on;

subplot(2,1,2);
stem(0:N_filt-1, h_filt(1:N_filt), 'filled', 'LineWidth', 1.2);
title('۴۰ نمونه filter پاسخ ضربه با');
xlabel('n'); ylabel('h[n]');
grid on;

```



دو روش خروجی یکسان دارند.

فعالیت ۱۱: برنامه‌ای بنویسید که پاسخ پله‌ی سیستم فوق را محاسبه و ۴۰ سمپل از آن را نمایش دهد.

```

b = [0.9, -0.45, 0.35, 0.2];
a = [1, 0.71, -0.46, -0.62];
N = 40;

```

```

% cumsum
h = impz(b, a, N);
s1 = cumsum(h);

```



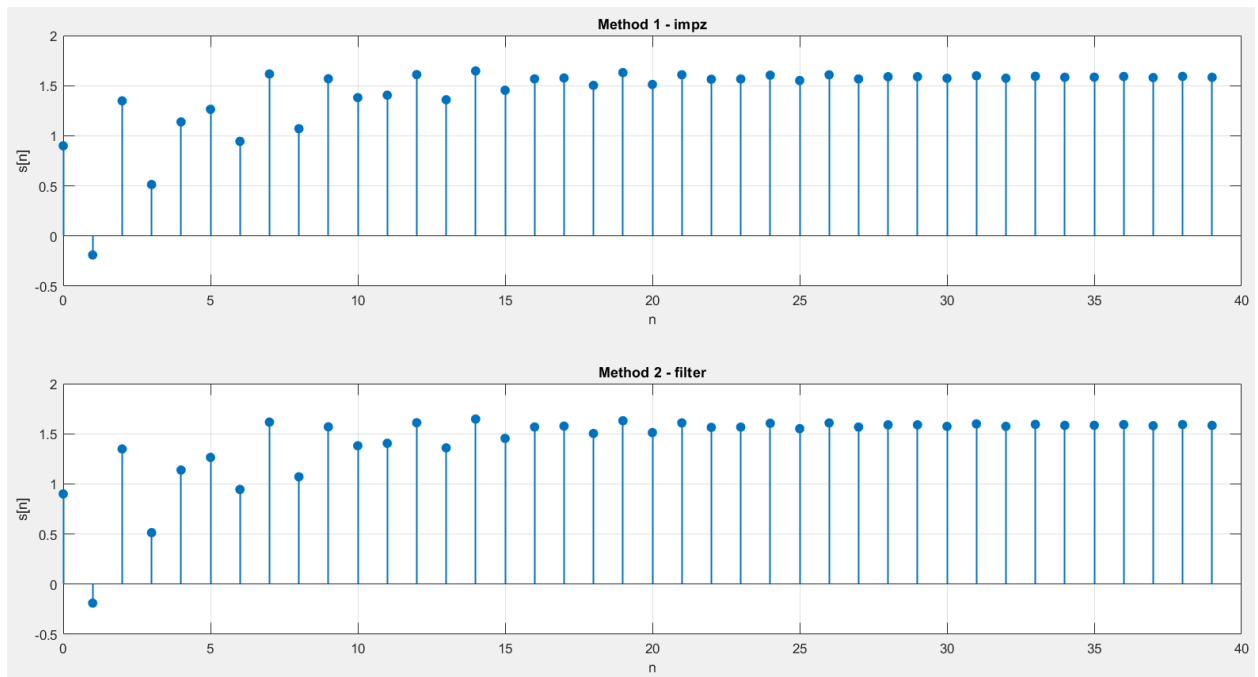
```

% filter
u = ones(1, N);
s2 = filter(b, a, u);

% رسم
figure;
subplot(2,1,1);
stem(0:N-1, s1, 'filled', 'LineWidth', 1.2);
title('Method 1 - impz');
xlabel('n'); ylabel('s[n]');
grid on;

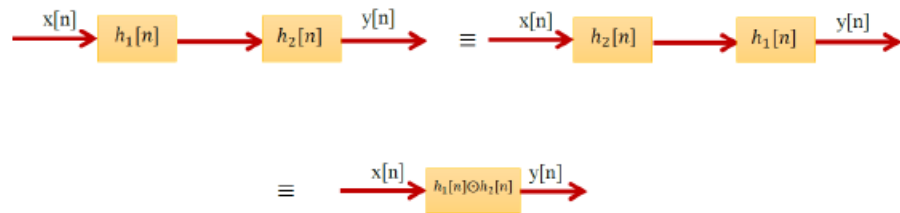
subplot(2,1,2);
stem(0:N-1, s2, 'filled', 'LineWidth', 1.2);
title('Method 2 - filter');
xlabel('n'); ylabel('s[n]');
grid on;

```



بخش ۵: سری کردن سیستم‌ها

در عمل می‌توان سیستم‌های علی و LTI مرتبه بالا را میتوان با استفاده از سری کردن سیستم‌های مرتبه پایین-تر پیاده‌سازی کرد. این کار چه مزیتی دارد؟ در شکل زیر اتصال سری را مشاهده میکنید.



فعالیت ۱۲: سیستم مرتبه ۴ زیر را پیاده سازی کرده

$$y[n] + 1.6y[n-1] + 2.28y[n-2] - 1.35y[n-3] + 0.68y[n-4] = 0.06x[n] - 0.19x[n-1] + 0.27x[n-2] - 0.26x[n-3] + 0.12x[n-4]$$

سپس نشان دهید که آیا میتوان این سیستم را با سری کردن دو سیستم زیر پیاده سازی کرد، یا خیر؟

$$y_1[n] + 0.9y_1[n-1] + 0.8y_1[n-2] = 0.3x[n] - 0.2x[n-1] + 0.4x[n-2]$$

$$y_2[n] + 0.7y_2[n-1] + 0.85y_2[n-2] = 0.2y_1[n] - 0.5y_1[n-1] + 0.3y_1[n-2]$$

محاسبات سری کردن دو سیستم به صورت زیر است:

$$H(z) = H_1(z) \cdot H_2(z) = \frac{B_1}{A_1} * \frac{B_2}{A_2}$$

بنابراین میتوانیم برای سری کردن از کانولوشن صورت ها و مخرج ها استفاده کنیم.

```
b = [0.06, -0.19, 0.27, -0.26, 0.12];
a = [1, 1.6, 2.28, -1.35, 0.68];
```

```
% sys 1
```

```
b1 = [0.3, -0.2, 0.4];
a1 = [1, 0.9, 0.8];
```

```
% sys 2
```

```
b2 = [0.2, -0.5, 0.3];
a2 = [1, 0.7, 0.85];
```

```
b_series = conv(b1, b2);
a_series = conv(a1, a2);
```

```
disp('ضرایب سیستم اصلی:');
disp('b = '); disp(b);
disp('a = '); disp(a);
```

```
disp('ضرایب سری دو سیستم:');
disp('b_{series} = '); disp(b_series);
disp('a_{series} = '); disp(a_series);
```

```

ضرایب سیستم اصلی:
b =
    0.0600    -0.1900    0.2700   -0.2600    0.1200

a =
    1.0000    1.6000    2.2800   -1.3500    0.6800

ضرایب سری دو سیستم:
b_{series} =
    0.0600    -0.1900    0.2700   -0.2600    0.1200

a_{series} =
    1.0000    1.6000    2.2800    1.3250    0.6800

```

خیر به دلیل اختلاف در a و a_series نمیتوانیم دو سیستم مرتبه ۲ (چیزی که سوال داده) را با هم سری کنیم و سیستم مرتبه ۴ را بدست آوریم. اما در حالت کلی، سیستم مرتبه ۴ را می‌توان با سری کردن دو سیستم مرتبه ۲ دیگر پیاده‌سازی کرد.

فعالیت ۱۳: این برنامه را برای ورودی سینوسی یک مرتبه با شرایط اولیه صفر و یک مرتبه با شرایط اولیه غیر صفر اجرا کنید و نتیجه خود را بیان کنید.

```

b_main = [0.06, -0.19, 0.27, -0.26, 0.12];
a_main = [1, 1.6, 2.28, -1.35, 0.68];

b1 = [0.3, -0.2, 0.4];
a1 = [1, 0.9, 0.8];

b2 = [0.2, -0.5, 0.3];
a2 = [1, 0.7, 0.85];

b_series = conv(b1, b2);
a_series = conv(a1, a2);

N = 100;
n = 0:N-1;
f = 0.05;
x = cos(2*pi*f*n);

% ===== حالت اول: شرایط اولیه صفر =====
y_main_zic = filter(b_main, a_main, x);
y_series_zic = filter(b_series, a_series, x);
diff_zic = y_main_zic - y_series_zic;

Z = [0.5; -0.3; 0.8; -0.2];

% اجرای سیستم اصلی با شرایط اولیه غیر صفر
[y_main_ic, ~] = filter(b_main, a_main, x, Z);
[y_series_ic, ~] = filter(b_series, a_series, x, Z);
diff_ic = y_main_ic - y_series_ic;

```

```

figure;
subplot(2,2,1);
stem(n, y_main_zic, 'b', 'filled', 'LineWidth', 1);
title('سیستم اصلی - شرایط اولیه صفر');
xlabel('n'); ylabel('y[n]'); grid on;

subplot(2,2,2);
stem(n, y_series_zic, 'r', 'filled', 'LineWidth', 1);
title('سیستم سری - شرایط اولیه صفر');
xlabel('n'); ylabel('y[n]'); grid on;

% حالت دوم: شرایط غیر صفر
subplot(2,2,3);
stem(n, y_main_ic, 'b', 'filled', 'LineWidth', 1);
title('سیستم اصلی - غیر صفر');
xlabel('n'); ylabel('y[n]'); grid on;

subplot(2,2,4);
stem(n, y_series_ic, 'r', 'filled', 'LineWidth', 1);
title('سیستم سری - غیر صفر');
xlabel('n'); ylabel('y[n]'); grid on;

% ===== نمایش مقایسه در یک پنجره =====
figure;

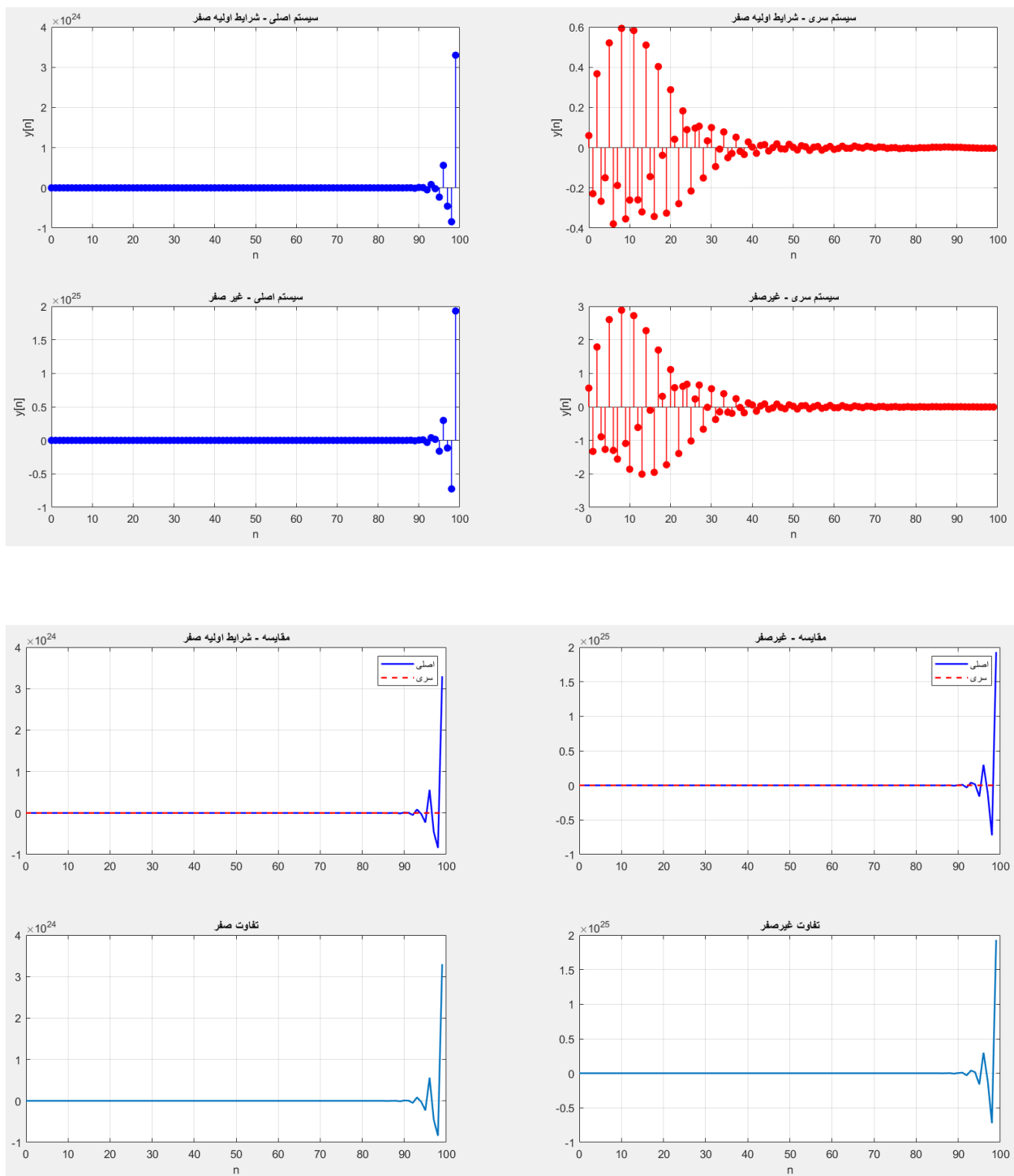
subplot(2,2,1);
plot(n, y_main_zic, 'b', n, y_series_zic, 'r--', 'LineWidth', 1.5);
legend('اصلی', 'سری');
title('مقایسه - شرایط اولیه صفر');
grid on;

subplot(2,2,2);
plot(n, y_main_ic, 'b', n, y_series_ic, 'r--', 'LineWidth', 1.5);
legend('اصلی', 'سری');
title('مقایسه - غیر صفر');
grid on;

subplot(2,2,3);
plot(n, diff_zic, 'LineWidth', 1.5);
title('تفاوت صفر');
xlabel('n'); grid on;

subplot(2,2,4);
plot(n, diff_ic, 'LineWidth', 1.5);
title('تفاوت غیر صفر');
xlabel('n'); grid on;

```



بخش ۶: کانولوشن

میدانیم که یکی از خواص سیستم‌های LTI این است که پاسخ سیستم را میتوان با استفاده از کانولوشن ورودی محدود با پاسخ ضربه سیستم بدست آورد. در این آزمایش این خاصیت را بررسی میکنیم.

فعالیت ۱۴: برنامه‌ای بنویسید که پاسخ سیستم قبل از به ورودی‌های مختلف ضربه، پله، سینوسی، یک مرتبه با استفاده از کانولوشن و یک مرتبه با استفاده از دستور *filter* بدست آورده و خروجی را در دو حالت مقایسه کند. آیا خروجی در دو حالت یکسان هستند؟ دلیل اضافه کردن صفر به ورودی قبل از فیلتر کردن چیست؟

```

b = [0.9, -0.45, 0.35, 0.2];
a = [1, 0.71, -0.46, -0.62];

N = 50;
n = 0:N-1;

% ۱) ورودی ضربیه
x_imp = [1, zeros(1, N-1)];

% ۲) ورودی پله
x_step = ones(1, N);

% ۳) ورودی سینوسی
x_sin = cos(2*pi*0.05*n);

% پاسخ ضربیه سیستم
h = impz(b, a, N)';

% filter
y_imp_filt = filter(b, a, x_imp);
y_step_filt = filter(b, a, x_step);
y_sin_filt = filter(b, a, x_sin);

% conv
y_imp_conv = conv(x_imp, h);
y_step_conv = conv(x_step, h);
y_sin_conv = conv(x_sin, h);

figure;
subplot(3,2,1);
stem(0:N-1, y_imp_filt, 'b', 'filled'); hold on;
stem(0:length(y_imp_conv)-1, y_imp_conv, 'r--');
title('ورودی ضربیه'); legend('filter', 'conv');
grid on;

subplot(3,2,2);
plot(0:N-1, y_imp_filt - y_imp_conv(1:N), 'LineWidth', 1.5);
title('تفاوت (ضربه)'); grid on;

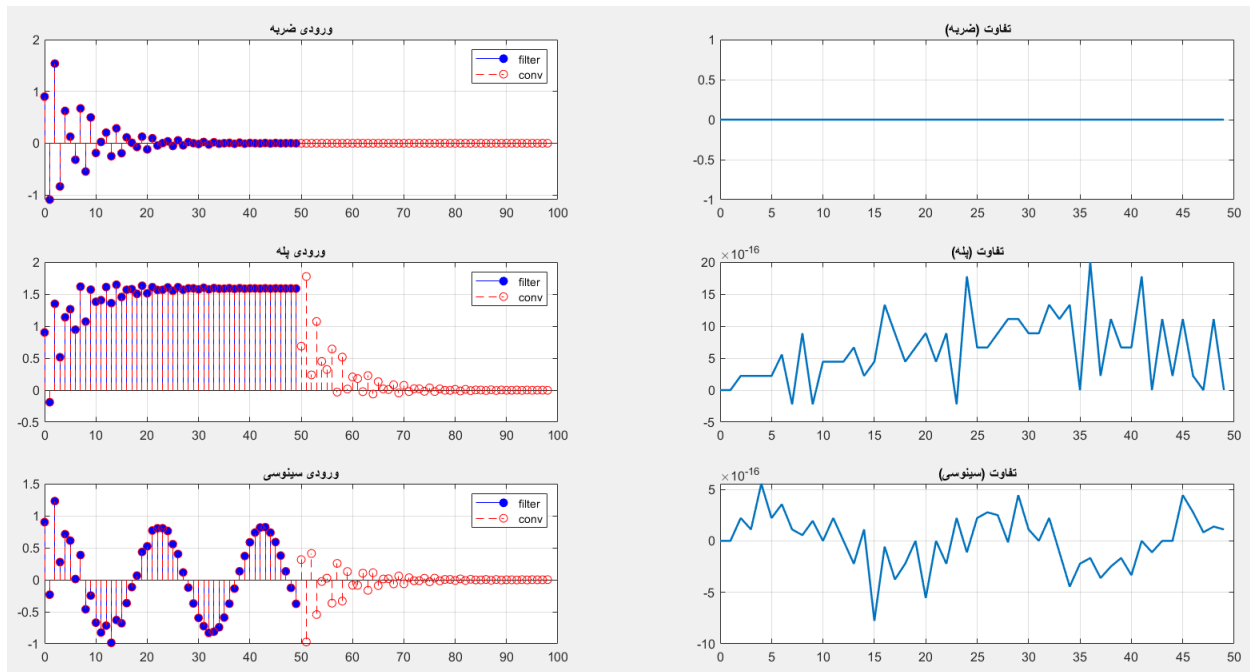
subplot(3,2,3);
stem(0:N-1, y_step_filt, 'b', 'filled'); hold on;
stem(0:length(y_step_conv)-1, y_step_conv, 'r--');
title('ورودی پله'); legend('filter', 'conv');
grid on;

subplot(3,2,4);
plot(0:N-1, y_step_filt - y_step_conv(1:N), 'LineWidth', 1.5);
title('تفاوت (پله)'); grid on;

subplot(3,2,5);
stem(0:N-1, y_sin_filt, 'b', 'filled'); hold on;
stem(0:length(y_sin_conv)-1, y_sin_conv, 'r--');
title('ورودی سینوسی'); legend('filter', 'conv');
grid on;

subplot(3,2,6);
plot(0:N-1, y_sin_filt - y_sin_conv(1:N), 'LineWidth', 1.5);
title('تفاوت (سینوسی)'); grid on;

```



همانطور که مشخص است اختلاف بسیار ناچیز در حد $e-16$ است و بنابراین هر دو روش پاسخ یکسانی به ما میدهند.

بخش ۷: بررسی پایداری سیستم‌ها

میدانیم که یک سیستم LTI زمانی نسبت به ورودی محدود پایدار است که پاسخ ضربه‌ی آن محدود باشد. بنابراین شرط لازم برای پایداری سیستم LTI و IIR این است که با افزایش اندیس زمان، پاسخ ضربه به سمت صفر همگرا شود.

شرط پایداری زیر را بررسی و اثبات کنید:

$$\sum_{n=-\infty}^{\infty} |h[n]| < \infty$$

فعالیت ۱۵: برای سیستم زیر خاصیت پایداری را بررسی کنید:

$$y[n] + 1.5y[n-1] + 0.9y[n-2] = x[n] - 0.8x[n-1]$$

```
b1 = [1, -0.8];
a1 = [1, 1.5, 0.9];

N = 100;
h1 = impz(b1, a1, N);

sum_abs_h1 = sum(abs(h1));

figure;
subplot(2,1,1);
```

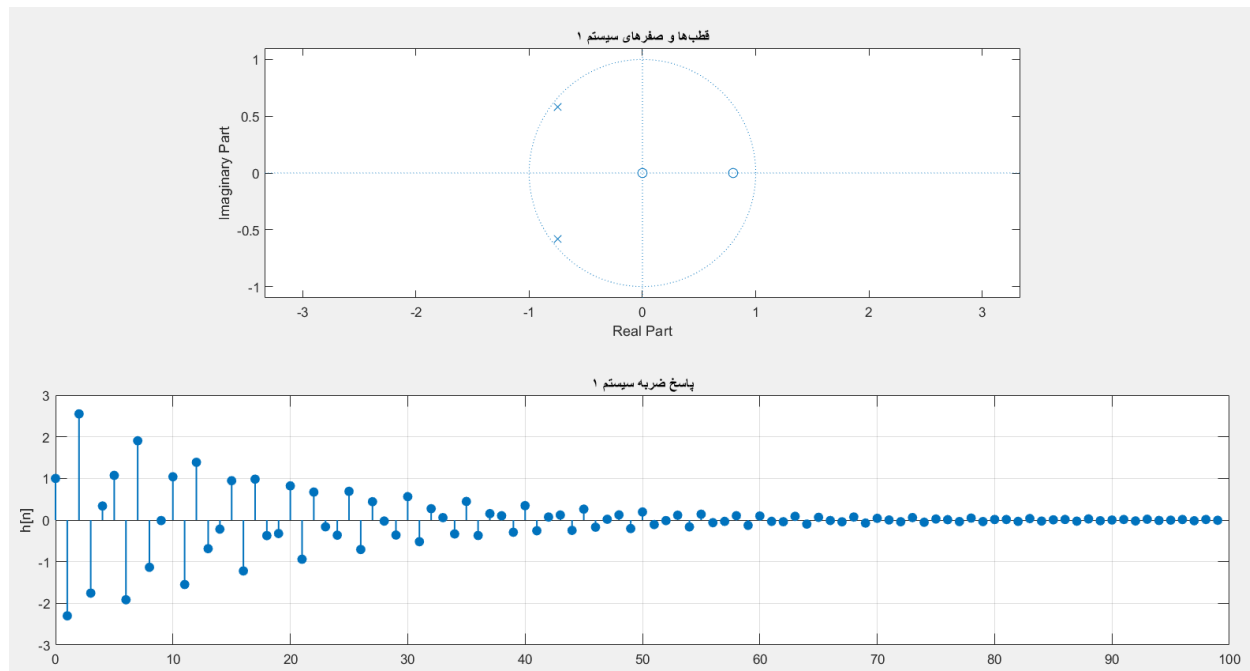
```

zplane(b1, a1);
title('قطب‌ها و صفرهای سیستم ۱');

subplot(2,1,2);
stem(0:N-1, h1, 'filled', 'LineWidth', 1.2);
title('پاسخ ضربه سیستم ۱');
xlabel('n'); ylabel('h[n]');
grid on;

disp(['Q15 - Sum of h[n]: ', num2str(sum_abs_h1)]);

```



این سیستم پایدار است - مجموع قدر مطلق پاسخ ضربه محدود داد - صفر ها و قطب ها داخل دایره واحد هستند.

Q15 - Sum of h[n]: 35.1782

فعالیت ۱۶: برای سیستم زیر خاصیت پایداری را بررسی کنید:

$$y[n] = x[n] - 4x[n-1] + 3x[n-2] + 1.76y[n-1] - y[n-2]$$

```

b2 = [1, -4, 3];
a2 = [1, -1.76, 1];

h2 = impz(b2, a2, N);
sum_abs_h2 = sum(abs(h2));

figure;
subplot(2,1,1);
zplane(b2, a2);
title('قطب‌ها و صفرهای سیستم ۲');

```

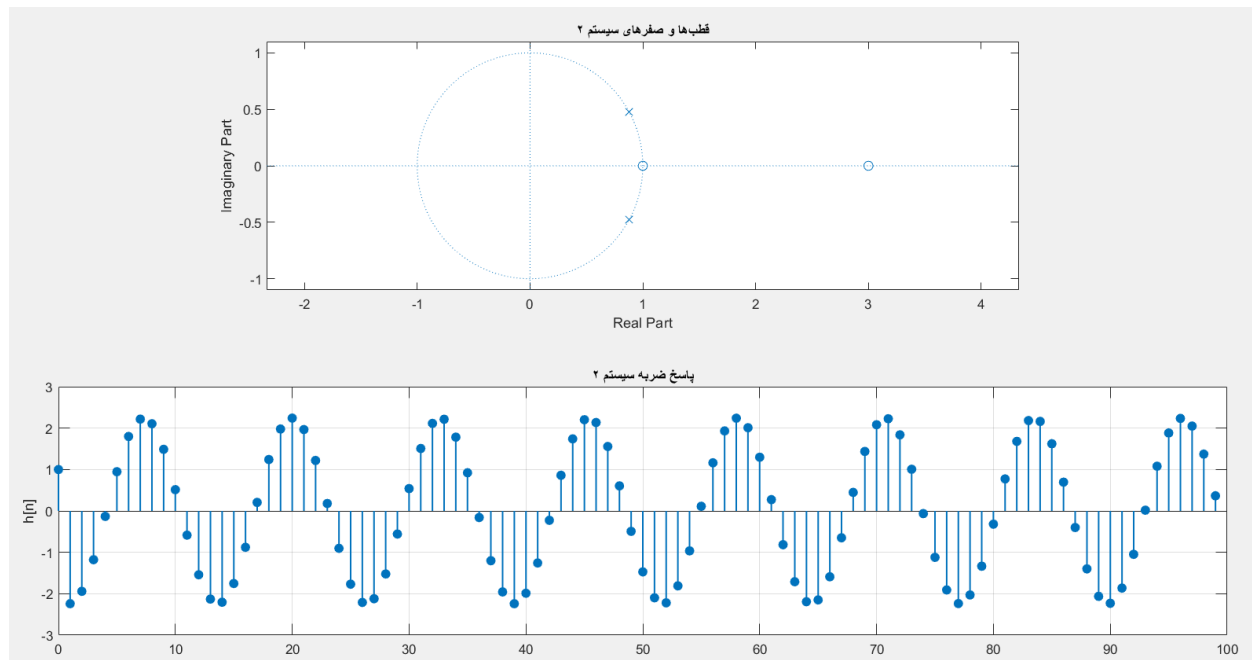


```

subplot(2,1,2);
stem(0:N-1, h2, 'filled', 'LineWidth', 1.2);
title('پاسخ ضربه سیستم ۲');
xlabel('n'); ylabel('h[n]');
grid on;

disp(['Q16 - Sum of h[n]: ', num2str(sum_abs_h2)]);

```



این سیستم ناپایدار است - مجموع قدر مطلق پاسخ ضربه بینهایت دارد - صفر و قطب بیرون دایره واحد دارد.

Q16 - Sum of $h[n]$: 142.3259